Recursion and Memoization

A Beginner-Friendly Editorial

A quick note before we start

If you haven't done Dynamic Programming before, that's completely fine. You don't really need to know DP to solve these questions.

The main idea is to look at the problem a little differently. Instead of trying to count everything at once, ask:

What could the last step have been?

Usually, once we fix the last step, the rest of the problem becomes a smaller version of the same problem.

We'll build this idea through the four questions.

1 Problem 1: Colored Tiling of a Strip

We have a $1 \times N$ strip. We can use:

- a 1×1 tile in 3 different colours, or
- a 1×2 tile in 1 colour.

Let $f(N)$ be the number of ways to completely tile a strip of length N.

The trick

Let's look at the last tile.

There are only two possibilities.

If the last tile has length 1, it can have any of the 3 colours. Before it, we have to tile a strip of length $N - 1$.

So this gives

$$3f(N-1)$$

possibilities.

If the last tile has length 2, there is only one choice for its colour. Before it, we need to tile a strip of length $N - 2$.

So this gives

$$f(N-2)$$

possibilities.

Putting the two cases together,

$$\boxed{f(N) = 3f(N-1) + f(N-2)}$$

That's the main observation for this question.
We also need some starting values. For an empty strip,

$$f(0) = 1,$$

and for a strip of length 1,

$$f(1) = 3.$$

Let's do one testcase

Let's find $f(4)$.
First,

$$f(2) = 3f(1) + f(0)$$

$$= 3(3) + 1 = 10.$$

Then,

$$f(3) = 3f(2) + f(1)$$

$$= 3(10) + 3 = 33.$$

So,

$$f(4) = 3f(3) + f(2)$$

$$= 3(33) + 10$$

$$\boxed{f(4) = 109}.$$

From here, there's nothing new to figure out. Just keep using the same formula.

Try calculating $f(5)$ and $f(6)$ yourself.

The important part of this question is figuring out the recurrence. Once you have that, the rest is just calculation.

2 Problem 2: Binary Clock

We need to count binary strings of length 12 which do not contain

$$000$$

or

$$111.$$

For example, 0010110 is valid, but 0011101 is not.

What should we keep track of?

We are building the string one bit at a time.

The only thing that can make a string invalid is getting three equal bits in a row.

So we don't need to remember the entire string. We only need to know what its ending looks like.

A valid string can end in something like

$$\ldots 0, \quad \ldots 00, \quad \ldots 1, \quad \ldots 11.$$

It cannot end in $\ldots 000$ or $\ldots 111$.

Let's use two counts:

$$A_n = \text{number of valid strings of length } n \text{ ending with one equal bit}$$

and

$$B_n = \text{number of valid strings of length } n \text{ ending with two equal bits.}$$

For example, 001 is counted by A_n, while 0011 is counted by B_n.

Finding the formulas

To get a string ending with one equal bit, we have to switch the previous bit.

So we can get it from either type of valid string of length $n-1$:

$$\boxed{A_n = A_{n-1} + B_{n-1}}.$$

For a string ending with two equal bits, the previous ending must have had only one equal bit. Otherwise, we'd create three equal bits.

Hence,

$$\boxed{B_n = A_{n-1}}.$$

For length 1, the strings are just

$$0, \quad 1.$$

Both have one equal bit at the end, so

$$A_1 = 2, \qquad B_1 = 0.$$

Let's do one small example

For length 2,

$$A_2 = A_1 + B_1 = 2 + 0 = 2$$

and

$$B_2 = A_1 = 2.$$

Therefore the total number of valid strings of length 2 is

$$A_2 + B_2 = 2 + 2 = \boxed{4}.$$

That's the whole process. We keep calculating the next pair of values from the previous pair.

Try calculating the values for lengths 3, 4, and 5 yourself. Then continue until length 12.

You should get

$$\boxed{466}$$

for length 12.

One thing to notice here is that we are saving the answers we already calculated instead of starting over every time. This is where the idea of Dynamic Programming starts showing up.

3 Problem 3: Weighted Ternary Strings

We have strings made using

$$0, \quad 1, \quad 2$$

where

$$w(0) = 1, \qquad w(1) = 1, \qquad w(2) = 2.$$

The string is valid only if it does not contain 22.

This time, we're interested in the **total weight** of the string, not its length.

What do we need to remember?

The only restriction is 22.

So, if our current string already ends in 2, we cannot add another 2.

That's all the information we need.

Let

$$A_n = \text{number of valid strings of weight } n \text{ ending in 0 or 1}$$

and

$$B_n = \text{number of valid strings of weight } n \text{ ending in 2.}$$

Adding 0 or 1

Both 0 and 1 have weight 1.

So to make a string of weight n ending in 0 or 1, we can take any valid string of weight $n-1$ and append either 0 or 1.

Therefore,

$$\boxed{A_n = 2(A_{n-1} + B_{n-1})}.$$

Adding 2

The digit 2 has weight 2.

So we need a valid string of weight $n-2$ before it.

However, that string cannot already end in 2, otherwise we would get 22.

Hence,

$$\boxed{B_n = A_{n-2}}.$$

We start with

$$A_0 = 1, \qquad B_0 = 0.$$

Let's do one example

Let's find the number of valid strings of weight 2.

First,

$$A_1 = 2(A_0 + B_0) = 2.$$

Also,

$$B_1 = 0$$

since a single 2 already has weight 2.

Now,

$$A_2 = 2(A_1 + B_1)$$

$$= 2(2) = 4.$$

And

$$B_2 = A_0 = 1.$$

So the total is

$$A_2 + B_2 = 4 + 1 = \boxed{5}.$$

From here, just keep applying the same two formulas.

Try calculating all the way up to weight 5 yourself. Then use the same process for weights 7 and 10.

The answers are

$$\boxed{N = 5: \ 66}$$

$$\boxed{N = 7: \ 368}$$

$$\boxed{N = 10: \ 4832}.$$

The main thing to notice here is that the quantity we're building on can be something other than string length. Here, it is the total weight.

4 Problem 4: Restricted Distance Jumps

We start at position 0 and can jump by

$$1, \quad 2, \quad 4.$$

There are two restrictions:

- A jump of 2 cannot immediately follow a jump of 1.
- A jump of 4 cannot immediately follow a jump of 2.

Why can't we just count positions?

Suppose we are at position 5.

That alone doesn't tell us what jumps are allowed next.

If the previous jump was 1, we cannot jump 2.

If the previous jump was 4, we can jump 2.

So this time, we need to remember what the previous jump was.

Let's define

$$D_1(n) = \text{ways to reach } n \text{ with the last jump being 1,}$$

$$D_2(n) = \text{ways to reach } n \text{ with the last jump being 2,}$$

and

$$D_4(n) = \text{ways to reach } n \text{ with the last jump being 4.}$$

Last jump is 1

A jump of 1 can follow any previous jump.

Therefore,

$$\boxed{D_1(n) = D_1(n-1) + D_2(n-1) + D_4(n-1)}.$$

Last jump is 2

A jump of 2 cannot follow a jump of 1.

So the previous jump must have been 2 or 4:

$$\boxed{D_2(n) = D_2(n-2) + D_4(n-2)}.$$

Last jump is 4

A jump of 4 cannot follow a jump of 2.

So the previous jump must have been 1 or 4:

$$\boxed{D_4(n) = D_1(n-4) + D_4(n-4)}.$$

The first jump has no restrictions, so our starting values include

$$D_1(1) = 1, \qquad D_2(2) = 1, \qquad D_4(4) = 1.$$

Let's do one testcase

Let's calculate the answer for $N = 7$.

Using the formulas above, we get

$$D_1(7) = 8,$$

$$D_2(7) = 1,$$

and

$$D_4(7) = 2.$$

These are three separate groups, so we add them:

$$8 + 1 + 2 = \boxed{11}.$$

That's our answer for $N = 7$.

Now try $N = 10$ and $N = 15$ yourself. The formulas stay exactly the same.

The answers are

$$\boxed{N = 10 : \ 37}$$

and

$$\boxed{N = 15 : \ 265}.$$

The main idea here is pretty useful: if what you are allowed to do next depends on what you did previously, you probably need to keep track of that previous choice.

One Last Thing

Across all four problems, the approach is basically:

1. Look at the last step.
2. Figure out what possibilities the last step has.
3. Turn each possibility into a smaller problem.
4. Save the answers to those smaller problems.
5. Use them to build the next answers.

Sometimes, knowing just the current position or length isn't enough.
Then we keep a little extra information:

- Q2: how many equal bits are at the end,
- Q3: whether the previous digit was 2,
- Q4: what the previous jump was.

This is basically the idea behind Dynamic Programming.
You don't need to worry too much about the terminology right now. The important part is getting used to asking:

What smaller answers can I reuse, and what information do I need to remember?

Once that starts feeling natural, DP becomes much less intimidating.